

FPGA/Verilog RTL 설계 경험서

Unified NTT RTL 사례로 배우는 Area와 Timing 설계

LUT, FF, CARRY4, DSP48E1, BRAM, fanout, routing,
pipeline, inference, interface, verification

JaeSeok Lee
Kookmin University
September 7, 2026

합성 보고서에서 출발해 RTL 구조로 해결하는 설계 기록

개정판을 읽는 방법

이 책은 문법별 장난감 예제를 나열하는 대신, 하나의 실제 설계가 100 MHz에서 150 MHz로 발전한 과정을 따라간다. 사례는 radix-2, radix-3, trinomial 변환을 지원하는 Unified NTT 가속기이며, 비교 대상은 다음 세 시점이다.

- 최초 100 MHz RTL: `old/rtl_verilog`
- 200 MHz 목표 실험판: `rtl_verilog_200mhz`
- 최종 MC V5 RTL: `rtl_verilog_mc`

중간 디렉터리의 “200mhz”는 목표 이름이다. 5.000 ns 제약에서 실제 timing은 달리지 않았으며 post-route WNS는 -3.563 ns였다. 최종판은 완전한 Zynq 시스템과 standalone RTL에서 150 MHz를 만족한다. 이 구분은 결과를 과장하지 않고, 실패한 시도까지 설계 판단의 근거로 남기기 위해 중요하다.

각 사례는 **관찰한 보고서**, **RTL에서 찾은 원인**, **선택한 구조**, **검증해야 할 계약**, **측정 결과** 순으로 설명한다. 코드 조각은 이해에 필요한 부분만 줄여 싣고, 정확한 신호 연결과 파라미터는 위 RTL 파일을 기준으로 한다.

수치 해석 원칙

서로 다른 device package, top-level, clock constraint의 결과를 단일 변수 실험처럼 해석하지 않는다. 이 책의 100 MHz와 150 MHz 수치는 실제 개발 milestone을 보여 주는 역사적 비교다. 인과관계는 matched run이 있는 경우에만 그 범위 안에서 말한다.

목차

개정판을 읽는 방법	i
1 RTL을 회로로 읽는 기준	1
1.1 조합식보다 경로를 먼저 그린다	1
1.2 FPGA 자원과 RTL의 대응	1
1.3 연산자의 실제 비용	1
1.3.1 덧셈과 뺄셈	2
1.3.2 mux와 variable select	2
1.3.3 constant shift와 constant multiply	2
1.4 RTL을 읽을 때의 다섯 질문	2
2 측정 범위와 보고서 읽기	3
2.1 먼저 같은 것을 비교하고 있는지 확인한다	3
2.2 재현 조건 표	3
2.3 Timing report의 읽는 순서	3
2.4 실패한 synthesis도 데이터다	4
2.5 Hierarchy report를 이용한 원인 분리	4
3 사례 설계의 계층과 발전 과정	5
3.1 처음부터 통합되어 있던 기능	5
3.2 100 MHz top과 최종 top의 차이	5
3.3 최종 core의 한 transaction 흐름	5
3.4 세 번의 설계 시점	6
3.5 최적화 동안 보존한 불변조건	6
4 Timing closure 실제 사례	7
4.1 사례 1: DUMP address에서 BRAM까지	7
4.1.1 첫 대응: address와 selection tag를 같이 등록한다	7
4.2 사례 2: DUMP를 고치자 LOAD가 병목이 되었다	8
4.3 최종 구조: 공유되는 registered host command	8
4.4 control reduction tree를 memory path에서 제거한다	9
4.5 writeback mapping을 한 cycle 앞당긴 physical tag	9
4.6 병목이 이동했는지 확인한다	9
5 150 MHz에 연산을 담기까지	11
5.1 성공한 회로만 보면 설계 이유가 사라진다	11
5.2 몇 ns가 걸리는지에 대한 직관	11
5.3 실패 1: subtract와 correction을 한 cycle에 연결했다	12
5.4 실패 2: ML-DSA product 결합과 digit 계산을 함께 수행했다	13

5.5	실패 3: Montgomery의 다섯 operand를 한 번에 더했다	13
5.6	실패 4: 상수곱에 stage 하나를 더 넣자 기능이 깨졌다	14
5.7	실패 5: 연산기를 최대한 공유했더니 더 커졌다	14
5.8	implementation directive만으로 150 MHz가 되지 않았다	15
6	Area와 inference 실제 사례	16
6.1	사례 1: BRAM inference가 먼저다	16
6.2	사례 2: logical depth와 physical depth를 분리한다	16
6.2.1	왜 정보가 사라지지 않는가	17
6.3	사례 3: scheme별 ROM을 word packing으로 통합한다	17
6.4	사례 4: pipeline metadata도 area다	17
6.5	사례 5: 계측 logic을 production에서 생성하지 않는다	18
6.6	Area 감소를 설명하는 올바른 문장	18
7	Verilog 연산자의 실제 자원 비용	19
7.1	연산자 하나를 같은 조건에서 합성해 본다	19
7.2	Addition과 subtraction: LUT가 carry를 구동한다	20
7.2.1	왜 modular subtraction이 무거운가	20
7.3	Comparator: 같은 23 bit라도 비용이 다르다	20
7.4	MUX: input 개수와 bit 폭을 함께 센다	20
7.5	Shift: constant와 variable은 전혀 다른 회로다	21
7.6	Multiplication과 reduction	21
7.7	면적을 예상하는 순서	21
8	공유 산술기와 pipeline 설계	23
8.1	설계 목표를 연산식과 자원으로 동시에 적는다	23
8.2	DSP0: 공통 low product	23
8.3	Narrow-prime path: R16 Montgomery multiplication	23
8.4	DSP1: mode별 곱과 정렬 산술 공유	24
8.5	공유 register는 live bit를 기준으로 설계한다	24
8.6	최종 correction addition 공유	25
8.7	ML-DSA path의 폭을 줄이는 방법	25
8.8	산술 timing의 현재 위치	25
9	Scheduler, memory mapping, interface	26
9.1	연산량보다 빈 cycle을 먼저 센다	26
9.2	NTRU+864의 half-empty issue 제거	26
9.3	Core-cycle 결과	26
9.4	AXI LOAD: beat 안의 병렬성을 memory lane으로 전달한다	27
9.5	AXI DUMP: backpressure보다 먼저 reservation한다	27
9.6	Interface 두 개를 비교하는 법	27
10	최적화를 깨지 않게 검증하는 방법	29
10.1	검증 대상은 값, cycle, protocol 세 층이다	29
10.2	latency를 하나의 상수로 관리한다	29
10.3	assertion으로 optimization 전제를 코드에 남긴다	29
10.4	변경 종류별 필요한 검증	30
10.5	실행 가능한 regression	30
10.6	simulation PASS와 implementation PASS를 구분한다	31

11 반복 가능한 최적화 workflow	32
11.1 한 iteration의 순서	32
11.2 Vivado에서 남길 최소 report	32
11.3 경로를 RTL 질문으로 번역한다	32
11.4 실험 기록 형식	33
11.5 이 사례에서 폐기해야 했던 단순 해석	33
11.6 최종 review checklist	34
12 측정 결과와 설계 결론	35
12.1 최종 세 범위의 post-route 결과	35
12.2 재현을 위한 post-synthesis 결과	35
12.3 100 MHz baseline과의 역사적 변화	35
12.4 자원별 area-time product	36
12.5 최종 end-to-end board latency	36
12.6 무엇이 실제로 개선을 만들었는가	37
A 용어 사전	38
B 자료와 재현 근거	39
B.1 AMD/Xilinx 문서	39
B.2 사례 RTL 파일	39
B.3 측정 report	39
B.4 개정 원칙	40

1. RTL을 회로로 읽는 기준

이 장의 결론

RTL 최적화의 단위는 코드 한 줄이 아니라 **register-to-register path와 저장 구조다**. 같은 연산자도 폭, 부호, fanout, 전후 mux, pipeline 위치에 따라 전혀 다른 회로와 배치 결과를 만든다.

1.1 조합식보다 경로를 먼저 그린다

동기식 경로 하나의 setup 조건은 다음처럼 생각할 수 있다.

$$T_{clk} \geq T_{cq} + T_{logic} + T_{route} + T_{setup} + T_{uncertainty} - T_{skew}.$$

RTL에서 직접 보이는 것은 주로 T_{logic} 의 후보뿐이다. FPGA에서는 LUT와 hard block이 물리적으로 떨어져 있고, 한 control net이 많은 load를 구동할 수 있으므로 T_{route} 가 더 클 수 있다. 따라서 긴 식을 짧게 보이도록 다시 쓰는 것만으로 timing이 해결된다고 가정하면 안 된다.

이 책의 100 MHz baseline 최악 경로가 좋은 예다. 논리 지연은 1.681 ns였지만 routing은 7.396 ns였다. 총 data path delay 9.077 ns 중 81.5%가 배선이었다. 이 경우의 설계 질문은 “식을 얼마나 줄일까”보다 “어느 register 경계에서 주소와 bank 결정을 끊을까”가 된다.

1.2 FPGA 자원과 RTL의 대응

자원	대표 RTL	설계자가 확인할 것
LUT6	Boolean logic, 작은 mux, decode	입력 수, logic level, 공유 신호의 fanout
FDRE	상태, pipeline, metadata	reset/enable, bit width, control set, 정렬 관계
CARRY4	덧셈, 뺄셈, 일부 비교	carry 길이, 앞뒤 mux, signed 확장
DSP48E1	곱셈과 곱셈 전후 산술	operand 폭, 내부 register, pre-adder/ALU 사용
RAMB36/18	동기 RAM/ROM/FIFO	depth와 width, port 동작, reset, inference template
LUTRAM/SRL	작은 RAM, FIFO, delay line	동기/비동기 read, reset이 inference를 막는지

자원 수와 속도는 독립 변수가 아니다. 예를 들어 register를 추가하면 FF는 증가하지만 조합 경로를 끊고 placement 자유도를 높일 수 있다. 반대로 여러 mode의 레지스터를 하나로 합치면 FF는 줄어도 입력 mux가 critical path에 붙을 수 있다. 선택은 항상 합성 후 경로와 hierarchy utilization으로 확인한다.

1.3 연산자의 실제 비용

1.3.1 덧셈과 뺄셈

고정 폭 덧셈은 보통 LUT와 carry chain으로 구현된다. 폭이 커질수록 carry가 길어지고, 입력 앞의 mode mux와 출력 뒤의 correction mux가 한 cycle에 함께 붙으면 경로가 깊어진다. 다음 두 표현은 수학적으로 간단하지만 하드웨어 질문은 서로 다르다.

```
1 sum = a + b;
2 out = (sum >= q) ? sum - q : sum;
```

확인할 항목은 (1) $a+b$ 의 범위, (2) 비교가 MSB/borrow로 흡수되는지, (3) correction이 같은 cycle인지, (4) q 가 상수인지다. 실제 범위가 한 번의 보정만 요구한다면 그 사실을 RTL 폭과 assertion에 남겨야 합성기와 검증 환경이 같은 전제를 공유한다.

1.3.2 mux와 variable select

여러 mode가 하나의 산술기를 공유하면 mux가 필요하다. mux를 산술기 입력에 두면 선택 지연 뒤에 carry/DSP가 이어지고, 결과 쪽에 두면 산술기가 복제될 수 있다. 어느 쪽이 좋은지는 “mux는 나쁘다”로 결정할 수 없다. 공유할 hard block, 허용 자원, critical path를 함께 본다.

최종 memory mapper는 generic variable bit-select 대신 실제 지원하는 tap만 case로 열거한다.

```
1 case (iMAP_SEL)
2   MAP_AFFINE4: side_w = addr[0] ^ addr[4];
3   MAP_AFFINE6: side_w = addr[0] ^ addr[6];
4   MAP_AFFINE7: side_w = addr[0] ^ addr[7];
5   default:     side_w = ^addr;
6 endcase
```

지원 공간을 정확히 표현하면 불필요한 barrel-select 구조를 피하고, 검증해야 할 경우의 수도 명확해진다.

1.3.3 constant shift와 constant multiply

고정 shift는 대부분 배선이다. 여러 shifted value의 합은 LUT와 carry chain을 소비한다. 따라서 “DSP를 쓰지 않았다”와 “면적이 작다”는 같은 말이 아니다. 최종 unifiedMUL의 narrow-prime 경로는 선택된 shift 값과 9-bit addition을 사용하지만, 큰 곱 T 와 uq 는 DSP에 남겨 둔다. 연산의 성격에 따라 hard multiplier와 shift/add를 나누는 사례다.

1.4 RTL을 읽을 때의 다섯 질문

실무 체크리스트

1. 이 값은 어느 register에서 출발해 어느 register 또는 hard-block pin에 도착하는가?
2. 경로에 LUT, carry, mux, variable select가 몇 단계 이어지는가?
3. 가장 긴 net의 fanout과 physical distance는 얼마인가?
4. 데이터와 valid, address, mode가 같은 cycle을 가리키는가?
5. 합성 결과가 의도한 BRAM, DSP, SRL로 inference되었는가?

이 다섯 질문은 뒤의 모든 실제 사례에서 반복 사용한다. 별도의 “좋은 코드 목록”을 외우기보다 보고서와 회로 계약을 연결하는 편이 재사용 가능하다.

2. 측정 범위와 보고서 읽기

2.1 먼저 같은 것을 비교하고 있는지 확인한다

“이 설계의 area”는 top-level을 말하지 않으면 불완전한 문장이다. 이 프로젝트는 다음 세 범위를 분리한다.

1. 완전한 AXI4-Stream/AXI DMA/SmartConnect/Zynq PS 시스템
2. 완전한 AXI BRAM Controller/SmartConnect/Zynq PS 시스템
3. standalone MDL RTL의 out-of-context 구현

첫 두 범위에는 processor interface와 data-movement IP가 들어간다. 세 번째는 가속기 RTL의 내부 경로와 자원을 보는 범위다. standalone 결과를 complete-system 결과와 나란히 놓을 수는 있지만, 서로 빼서 “AXI 비용”이라고 단정해서는 안 된다. hierarchy boundary와 최적화 조건이 다르기 때문이다.

2.2 재현 조건 표

PPA 결과를 기록할 때 적어도 다음 항목을 표 위에 둔다.

항목	기록 이유
Tool/version	inference와 physical optimization이 버전에 따라 달라진다.
Device/package/speed grade	같은 FPGA family라도 pinout과 배치 가능 영역, 속도가 달라진다.
Top-level/hierarchy	interface IP와 debug logic 포함 범위를 결정한다.
Clock constraint	utilization과 timing은 목표 주기에 반응한다.
Synthesis와 implementation directive	retiming, replication, placement 결과에 영향을 준다.
Post-synthesis와 post-route	전자는 구조 확인, 후자는 실제 routing을 포함한 최종 판단에 쓴다.

최종 세 범위는 Vivado 2020.2, xc7z020c1g484-1, 6.666 ns constraint로 맞췄다. 이 조건이 일치하므로 interface별 complete-system 결과를 비교할 수 있다.

2.3 Timing report의 읽는 순서

1. **Clock/path group**: 의도한 clock의 synchronous path인지 본다.
2. **Source/destination**: RTL의 어느 경계인지 찾는다.
3. **WNS/TNS/failing endpoints**: 최악 정도와 문제의 폭을 구분한다.

4. **Logic levels**: LUT와 CARRY4가 얼마나 이어지는지 본다.
5. **Logic/route 비율**: Boolean 재작성과 register/placement 중 우선순위를 정한다.
6. **Fanout**: 긴 route가 한 신호의 넓은 분배 때문인지 확인한다.

WNS가 양수이면 해당 제약을 만족한다. TNS가 0이고 failing endpoint가 0인지도 함께 확인한다. 다음 식으로 slack에서 대략적인 주파수를 계산할 수 있다.

$$T_{achieved} \approx T_{constraint} - WNS, \quad f_{approx} \approx \frac{1}{T_{achieved}}.$$

이 값은 현재 placement에서의 근사치다. 다른 constraint로 다시 구현했을 때 같은 placement가 유지된다는 보장은 없다.

2.4 실패한 synthesis도 데이터다

초기 baseline synthesis에서는 coefficient RAM이 BRAM으로 inference되지 않고 register로 풀렸다. 결과는 LUT 563,561개, FF 191,202개, BRAM 0개, WNS-9.387 ns였다. 이 숫자를 정상 architecture의 PPA로 사용하면 안 된다. 그러나 실패 원인을 찾는 데는 매우 강한 증거다. 보고서의 RAM “mem_reg” dissolved into registers와 write-process 경고가 먼저 수정해야 할 대상이 RAM coding template임을 알려 주었다.

이후 inference가 정상화된 100 MHz routed baseline은 LUT 5,278개, FF 3,072개, BRAM tile 7.5개, DSP48E1 4개가 되었다. area 최적화는 항상 이런 “정상 mapping baseline”을 확보한 뒤 시작해야 한다.

흔한 오관

BRAM inference 실패로 수십만 LUT가 나온 run과 정상 run의 감소율을 설계 개선율로 쓰지 않는다. 전자는 기능 구조가 물리 자원에 잘못 mapping된 진단 결과다.

2.5 Hierarchy report를 이용한 원인 분리

top utilization 한 줄만 보면 증가한 자원이 core 때문인지 interface 때문인지 알 수 없다. 최종 AXI4-Stream complete system 안의 md1 hierarchy는 4,739 LUT와 2,792 FF였고, standalone MDL은 4,736 LUT와 2,792 FF였다. 3-LUT 차이는 IP 경계를 넘는 최적화 수준의 작은 차이다. 이 일치는 standalone 결과가 통합 시스템 속 가속기와 같은 RTL을 대표한다는 확인 자료가 된다.

반면 complete-system BRAM은 stream 시스템 9 tile, BRAM-controller 시스템 6 tile이다. 이 차이는 arithmetic core가 아니라 DMA payload FIFO를 포함하는 interface 구성에서 나온다. hierarchy report는 이런 귀속을 가능하게 한다.

3. 사례 설계의 계층과 발전 과정

3.1 처음부터 통합되어 있던 기능

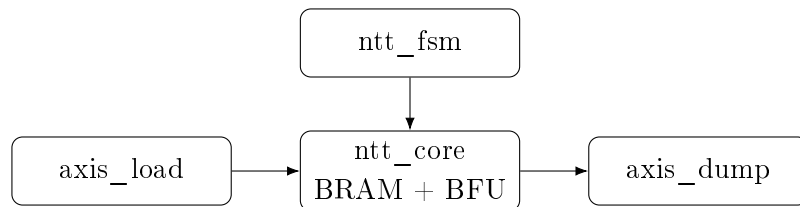
최초 100 MHz RTL은 이미 하나의 unifiedBUT에서 다음 연산을 선택했다.

- radix-2 Cooley–Tukey NTT와 Gentleman–Sande INTT
- radix-3 NTT와 INTT
- forward/inverse trinomial transform

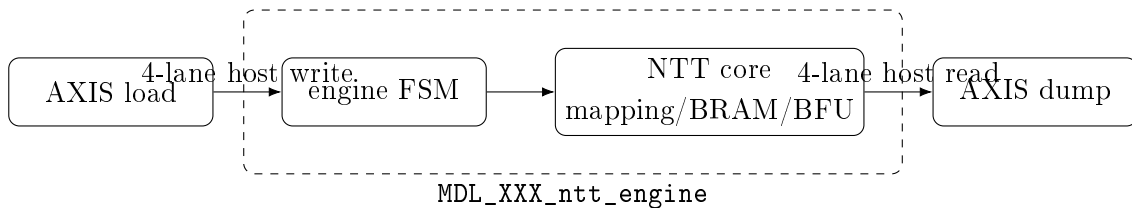
따라서 발전의 핵심을 “radix-2와 radix-3를 처음 통합했다”고 설명하면 사실과 다르다. 실제 개선 대상은 memory organization, address/control path, multiplier sharing, pipeline alignment, scheduler와 transport 였다. 최종판에는 Falcon modular NTT/INTT mode도 추가되었다.

3.2 100 MHz top과 최종 top의 차이

최초 top에는 AXI load, FSM, dump, core가 직접 나란히 있었다.



최종판은 transport-independent engine을 경계로 둔다.



AXI4-Stream top과 AXI BRAM Controller frontend가 같은 engine을 사용하므로, interface 실험이 arithmetic schedule을 조용히 바꾸지 않는다. 이 분리는 직접적인 LUT 감소 기법이라기보다 비교의 신뢰성과 재사용성을 높이는 architecture 결정이다.

3.3 최종 core의 한 transaction 흐름

1. ntt_addrgen이 logical coefficient address와 zeta address를 만든다.
2. layout_mapper가 logical address를 physical side와 compact row로 바꾼다.

3. 네 개의 true-dual-port coefficient BRAM에서 피연산자를 읽는다.
4. 두 zeta를 packed ROM에서 동기식으로 읽는다.
5. unifiedBUT이 선택된 radix/mode의 연산을 수행한다.
6. valid/last와 physical tag가 13-stage result에 맞춰 이동한다.
7. 반대 ping-pong set의 BRAM에 결과를 쓴다.

이 흐름의 중요한 제약은 연산 중 mode가 고정되고, 새로운 transaction을 매 cycle 받을 수 있다는 점이다. latency는 길어질 수 있지만 $\Pi=1$ 을 유지하면 layer의 steady-state 처리율은 유지된다.

3.4 세 번의 설계 시점

시점	관찰	다음 판단
100 MHz baseline	timing 충족, memory address/control route가 최악 경로	memory pin 앞 조합 경로를 register로 분할
200 MHz 목표판	DUMP 병목 제거 후 LOAD가 8-LUT/fanout-95 병목으로 이동, WNS-3.563 ns	한 경로의 patch가 아니라 host memory command 전체를 재구성
MC V5 150 MHz	compact memory, physical tag, shared host command, packed zeta, 재구성된 multiplier	세 interface/standalone 범위에서 timing과 area 검증

이 과정은 timing closure가 “한 번 pipeline을 넣고 끝내는 일”이 아님을 보여 준다. 한 병목을 제거하면 다음 실제 병목이 보인다. 매 iteration에서 새 report를 읽고, 원인 범위를 더 넓히거나 좁혀야 한다.

3.5 최적화 동안 보존한 불변조건

- 두 unifiedMUL은 총 네 DSP48E1을 사용한다.
- mode가 고정된 동안 multiplier와 butterfly는 매 cycle 새 입력을 받는다.
- ping-pong layer는 읽는 set과 쓰는 set을 분리한다.
- logical coefficient 순서와 NTT/INTT 수학 결과는 유지한다.
- AXI backpressure 중 전송된 beat를 잃거나 중복하지 않는다.

최적화 전에 이 불변조건을 먼저 적어 두면 “합성은 좋아졌지만 protocol 또는 cycle alignment가 바뀐” 회귀를 빠르게 찾을 수 있다.

4. Timing closure 실제 사례

4.1 사례 1: DUMP address에서 BRAM까지

100 MHz standalone baseline의 최악 setup path는 다음 경계였다.

```
u_dump/oDBG_RD1_ADDR_reg[2]
-> u_core/u_bram3/.../ADDRARDADDR[6]
```

항목	값	해석
Requirement	10.000 ns	100 MHz clock
Data path delay	9.077 ns	setup 여유가 매우 작음
Logic delay	1.681 ns	전체의 18.5%
Route delay	7.396 ns	전체의 81.5%
Logic levels	8	LUT2/3/4/5와 LUT6 네 개
Post-route WNS	+0.273 ns	100 MHz는 충족

주소는 layout side와 bank를 계산하고, LOAD/DUMP/CORE가 공유하는 BRAM port mux를 통과했다. logical address, bank selection, phase selection이 RAM address pin 앞 한 cycle에 모인 구조다. 200 MHz의 5 ns budget에는 맞지 않는다.

4.1.1 첫 대응: address와 selection tag를 같이 등록한다

중간 200 MHz 목표판은 DUMP address를 두 번 등록하고, bank/port tag도 BRAM 응답 cycle에 맞춰 등록했다. 핵심은 데이터만 늦추는 것이 아니라 그 데이터를 선택할 metadata도 같이 늦추는 것이다.

```
1 dbg_addr_q  <= iDBG_RD_ADDR;
2 dbg_addr_q2 <= dbg_addr_q;
3 dbg_bank_q  <= dbg_bank;
4 dbg_bank_q2 <= dbg_bank_q;
5
6 case (dbg_bank_q)
7   2'd0: begin bram0_ena = 1'b1; bram0_addra = dbg_addr_q2; end
8   2'd1: begin bram1_ena = 1'b1; bram1_addra = dbg_addr_q2; end
9   // ...
10 endcase
```

BRAM은 주소를 받은 다음 cycle에 동기식 read data를 내므로, output mux의 bank tag는 address command보다 한 단계 뒤에 있어야 한다. 이 변경 때문에 AXI dump valid pipeline도 3-stage에서 4-stage로 늘려 response data와 valid/TLAST를 다시 맞췄다.

pipeline 삽입의 실제 비용

주소 register 하나를 넣는 변경은 주소만의 변경이 아니다. bank, port, valid, last, FIFO reservation과 phase done 시점까지 같은 transaction을 가리키는지 확인해야 한다. 기능 simulation이 필요한 이유가 바로 이 cycle contract다.

4.2 사례 2: DUMP를 고치자 LOAD가 병목이 되었다

DUMP 경로를 약 1 ns 수준으로 줄인 뒤, 200 MHz 목표판의 최악 경로는 LOAD address에서 BRAM write-address pin으로 이동했다.

항목	값	해석
Data path delay	7.913 ns	5 ns budget 초과
Logic delay	1.448 ns	18% 수준
Route delay	6.465 ns	81%, placement/routing 지배
Logic levels	8 LUT	bank/port/phase address mux
문제 net fanout	95	넓은 memory control 분배
Post-route WNS	-3.563 ns	200 MHz 미충족

이 결과에서 얻은 교훈은 “register를 더 넣자”보다 구체적이다. LOAD와 DUMP가 각자 mapper, bank rail, port rail을 만들고 마지막에 phase mux로 합쳐지면 RAM 주변에 넓은 조합 cone과 high-fanout net이 생긴다. 따라서 최종판은 host transaction을 physical command로 먼저 바꾼 후 하나의 register bank에 저장했다.

4.3 최종 구조: 공유되는 registered host command

최종 MDL_XXX_ntt_core.v의 핵심 register는 다음과 같다.

```

1 reg      host_write_q;
2 reg [3:0] host_valid_q;
3 reg [3:0] host_side_q;
4 reg      host_set_q;
5 reg [LOCAL_ADDR_W-1:0] host_row0_q, host_row1_q,
6                      host_row2_q, host_row3_q;
7 reg [D-1:0] host_data0_q, host_data1_q,
8                      host_data2_q, host_data3_q;
```

LOAD와 DUMP는 동시에 발생하지 않는 phase다. mapper 출력인 side와 compact row를 이 register에 저장하고, BRAM port mux는 등록된 physical command만 본다. 이 경계는 세 가지를 동시에 해결한다.

1. logical-to-physical mapping을 BRAM pin 앞 경로에서 분리한다.
2. LOAD/DUMP가 각각 소유하던 physical command rail을 공유한다.
3. phase control의 큰 fanout을 register boundary에서 제한한다.

4개씩 정렬된 host transaction에서 port pattern은 지원 mapping 모두 A,A,B,B로 고정된다. 따라서 세 lane의 port 선택은 state register가 아니라 상수 wire가 된다.

```

1 wire host_port1_q = 1'b0;
2 wire host_port2_q = 1'b1;
```

```
3 wire host_port3_q = 1'b1;
```

동작 범위에서 항상 참인 불변조건을 찾아 runtime control을 없앤 사례다. 이 불변조건은 layout-mapper regression으로 모든 address에 대해 확인한다.

4.4 control reduction tree를 memory path에서 제거한다

최초 core는 pipeline이 비었는지 다음처럼 계산했다.

```
1 wire pipe_nonempty = |valid_pipe;
2 assign oBUSY = iSTART | ag_running | req_valid_q |
3             rd_valid_q | cap_valid_q | pipe_nonempty;
4 wire load_req = iLOAD_WE && !oBUSY;
```

긴 valid vector의 reduction OR가 oBUSY와 LOAD guard를 거쳐 BRAM address mux까지 영향을 주었다. 중간판과 최종판은 입출 수를 세는 작은 occupancy counter를 사용한다.

```
1 reg [3:0] pipe_cnt;
2 wire pipe_nonempty = (pipe_cnt != 4'd0) | rd_valid_q;
3
4 if (but_enable) begin
5     case ({rd_valid_q, wb_valid_int})
6         2'b10: pipe_cnt <= pipe_cnt + 4'd1;
7         2'b01: pipe_cnt <= pipe_cnt - 4'd1;
8     endcase
9 end
```

counter는 FF와 작은 adder를 추가한다. 대신 wide reduction tree가 매 cycle의 memory control을 구동하지 않게 한다. timing을 위해 약간의 local state를 쓰는 trade-off다.

최종 LOAD path에서는 !oBUSY 조건도 제거했다. top-level phase FSM이 LOAD와 CORE를 상호 배타적으로 보장하므로 core 내부에서 같은 조건을 다시 계산할 필요가 없다. 이 제거는 안전성 전제를 hierarchy 계약으로 올린 것이다. 따라서 top-level FSM과 host write assertion을 함께 검증해야 한다.

4.5 writeback mapping을 한 cycle 앞당긴 physical tag

최초판은 logical address를 pipeline 끝까지 운반한 뒤 writeback 시점에 layout과 bank를 다시 계산했다. 최종판은 read request에서 {side,row} tag를 만들고 result와 같은 길이로 이동시킨다.

```
1 req_tag_a_q <= {rd_side_a, rd_row_a};
2 // ... synchronous BRAM read and butterfly pipeline ...
3 wb_side_a_pre = a_pipe[PIPE_STAGES-2][ADDR_W-1];
4 wb_row_a_pre  = a_pipe[PIPE_STAGES-2][LOCAL_ADDR_W-1:0];
```

ping-pong set bit만 반전하면 최종 physical bank가 결정되므로 writeback pin 앞에 parity/affine mapping logic을 다시 둘 필요가 없다. “계산 결과”뿐 아니라 “어디에 쓸 결과인가”도 pipeline data로 취급한 설계다.

4.6 병목이 이동했는지 확인한다

최종 150 MHz 결과에서 최악 경로는 더 이상 DUMP/LOAD address에서 BRAM pin으로 가는 경로가 아니다.

범위	최악 경로 종류	Data delay	WNS
Standalone MDL	butterfly mode에서 modular add register	6.522 ns	+0.089 ns
Complete AXI4-Stream	address-generator state에서 request tag	6.366 ns	+0.075 ns
Complete BRAM controller	butterfly input에서 constant-multiply register	6.550 ns	+0.006 ns

이 표는 모든 경로가 넉넉하다는 뜻이 아니다. BRAM complete system은 6.666 ns constraint에 0.006 ns만 남는다. 다만 처음의 memory-address 구조가 제거되어 다음 optimization 대상이 scheduler와 arithmetic 내부로 이동했음을 보여 준다.

실무 체크리스트

- pipeline을 넣기 전 source/destination과 route 비율을 기록한다.
- address와 data를 선택하는 tag/valid/last를 같은 transaction으로 이동한다.
- 수정 후 이전 경로가 아니라 새 top- N 경로를 다시 읽는다.
- high-fanout constraint를 넣기 전에 구조적으로 load 수를 줄일 수 있는지 본다.
- positive WNS뿐 아니라 zero failing endpoint와 hold도 확인한다.

5. 150 MHz에 연산을 담기까지

5.1 성공한 회로만 보면 설계 이유가 사라진다

최종 RTL만 읽으면 register가 왜 그 위치에 있는지 알기 어렵다. 이 설계에서는 clock constraint를 바꾸고 top timing path를 다시 읽을 때마다 병목의 종류가 달라졌다. 남아 있는 report와 RTL milestone을 시간 순서로 놓으면 다음과 같다.

시점	관찰한 경로와 결과	다음 판단
최초 100 MHz	DUMP address → BRAM address, 9.077 ns	주소, bank, phase 선택을 BRAM pin 앞에서 끝내는 구조는 더 높은 주파수에 맞지 않았다.
초기 150 MHz	mode select → radix-3 subtraction register, WNS = -0.338 ns	한 cycle에 연결된 23-bit carry 연산과 뒤쪽 선택을 분리해야 했다.
200 MHz 산술 실험	DSP product → ML-DSA intermediate register, 8.168 ns	product 결합과 reduction digit 계산을 한 register 경계 안에 두지 않기로 했다.
최종 150 MHz	standalone, AXI4-Stream, BRAM complete system 모두 timing 충족	목표 주파수를 검증된 production point 로 고정하고 세 범위를 따로 보존했다.

이 표의 각 행은 동일한 단일 변수 실험이 아니다. 개발 중 서로 다른 snapshot을 나열한 설계 일지다. 따라서 수치 차이 전체를 특정 RTL 변경의 효과로 해석하지 않는다. 여기서 사용할 수 있는 증거는 “그 시점의 top path가 무엇이었는데가”와 “그 path의 조합 구조를 다음 RTL에서 어떻게 끊었는가”다.

5.2 몇 ns가 걸리는지에 대한 직관

RTL 연산 이름만으로 지연을 고정값처럼 외우면 안 된다. 같은 23-bit addition도 앞의 operand MUX, 뒤의 result MUX, fanout과 배치 위치에 따라 route delay가 달라진다. 이 프로젝트에서 실제 관찰한 path를 구성별로 나누면 다음과 같다.

실제 경로	Logic	Route	Total	관찰한 구조
최초 DUMP address → BRAM	1.681 ns	7.396 ns	9.077 ns	8 LUT, route 81.5%
초기 radix-3 subtraction result	2.896 ns	3.781 ns	6.677 ns	11 CARRY4를 포함한 13 levels
200 MHz ML-DSA intermediate	4.600 ns	3.568 ns	8.168 ns	12 CARRY4를 포함한 17 levels

최종 standalone modular-add register	2.494 ns	4.028 ns	6.522 ns	4 CARRY4를 포함한 7 levels
최종 BRAM system 1006x table-sum	2.920 ns	3.630 ns	6.550 ns	6 CARRY4를 포함한 10 levels

여기서 얻는 실무적 직관은 세 가지다.

1. 23-bit carry 연산 두 개와 mode selection이 이어진 path는 이 implementation에서 6.677 ns였고 150 MHz에 들어가지 않았다.
2. 더 복잡한 product 결합과 field 계산은 8.168 ns였다. 5 ns에 맞추려면 식 정리만으로 부족하고 register 또는 DSP 내부 stage가 필요했다.
3. logic이 짧아도 BRAM 주변처럼 route가 80%를 넘으면 9 ns가 될 수 있다. 반대로 “adder는 몇 ns”라는 표만으로 system timing을 예측할 수 없다.

이 수치는 xc7z020 계열, Vivado 2020.2, 해당 placement와 constraint에서 얻은 경로 측정값이다. 다른 device, speed grade, floorplan과 fanout에는 그대로 대입하지 않는다. 설계 초기에 필요한 것은 절대값 암기가 아니라 “carry가 몇 번 직렬인가”, “MUX가 carry 앞뒤 어디에 있는가”, “logic보다 route가 큰가”를 구분하는 감각이다.

5.3 실패 1: subtract와 correction을 한 cycle에 연결했다

modular subtraction은 식으로 쓰면 짧다.

$$d = a - b, \quad r = \begin{cases} d + q, & d < 0, \\ d, & d \geq 0. \end{cases}$$

그러나 23-bit RTL에서 raw subtraction, borrow 선택, conditional addition과 mode-dependent result 선택이 같은 register 앞에 놓이면 carry chain 두 개와 MUX가 직렬로 합성될 수 있다.

초기 150 MHz post-route report의 top path는 mode register에서 radix-3 subtraction result register까지였다. 6.667 ns requirement에서 data path가 6.677 ns였고, logic 2.896 ns와 route 3.781 ns를 사용했다. 13 logic levels 가운데 CARRY4가 11개였다. setup slack은 -0.338 ns였다. 이 결과는 “subtractor 하나가 느리다”보다 다음 사실을 보여 준다.

- carry 결과를 고르는 mode MUX도 같은 path의 일부다.
- 산술 depth와 route가 모두 커서 placement directive만으로 해결하기 어렵다.
- source가 mode register라는 사실은 연산뿐 아니라 control fanout도 함께 줄여야 함을 뜻한다.

최종 RTL은 모든 modular subtraction에 새 stage를 추가하지 않았다. multiplier 앞에서 일찍 필요한 두 general difference만 C1과 C2로 나눴다.

```

1 // C1: raw subtraction and borrow
2 raw0_c1    <= {1'b0, b} - {1'b0, a};
3 borrow0_c1 <= raw0[D];
4
5 // C2: conditional correction
6 corrected = {1'b0, raw0_c1[D-1:0]} + {1'b0, q};
7 d0_c2     <= borrow0_c1 ? corrected[D-1:0]
8           : raw0_c1[D-1:0];

```

C2는 이 rail에서 원래 alignment에 쓰이던 자리였다. 그러므로 새 latency를 더하지 않고 기존 register의 일을 바꾸어 두 carry chain을 서로 다른 cycle에 배치했다. 이 사례에서 핵심 판단은 “pipeline을 한 단계 늘린다”가 아니라 **이미 존재하는 alignment stage에 실제 연산을 배치한다**는 것이다.

또한 arithmetic result인 radix-3의 E 와 S 를 첫 register에서 공통 A/C rail에 바로 합치지 않았다. carry chain 직후에 mode MUX가 붙기 때문이다. 첫 register는 분리하고, 다음 경계에서 이미 등록된 값끼리 merge했다.

5.4 실패 2: ML-DSA product 결합과 digit 계산을 함께 수행했다

200 MHz 실험의 한 post-route top path는 DSP48E1의 product output에서 a_tmp_buf register까지였다. 5.000 ns requirement에 data path는 8.168 ns, logic levels는 17개였고 그중 CARRY4가 12개였다. slack은 -3.171 ns였다.

당시 조합 cone은 low product의 정렬된 high part와 다른 partial product를 더한 뒤, 그 결과에서 여러 field sum을 연속으로 만들었다. RTL의 수식은 맞고 simulation도 통과했지만, Vivado가 본 물리 경로에는 긴 carry chain이 남았다. “중간 wire 이름을 여러 개 둔다”는 것은 pipeline이 아니다. register edge가 없으면 모든 식은 여전히 한 cycle의 경로다.

최종 six-stage unifiedMUL은 일을 다음처럼 나눈다.

1. DSP0가 16-bit low product를 계산한다.
2. DSP1의 MREG/PREG가 ML-DSA high partial product와 aligned cross-sum을 받아 외부 fabric의 긴 product-adder 경계를 줄인다.
3. C3에서 $H_0 + H_1$ 을 한 번 계산하고 다음 high-part 식에서도 재사용한다. 별도 24-bit addition 대신 두 번째 carry chain을 14 bit로 제한한다.
4. C4에서 segmented residual, C5에서 base/correction addition, C6에서 constant-selected canonical correction을 수행한다.

마지막 correction도 high digit과 low digit을 각각 고치는 branch로 표현했을 때 합성기가 branch MUX와 carry를 깊게 합쳤다. 최종판은 범위 판정으로 0, $+q$, $-q$, $-2q$ 중 correction constant 하나를 고르고 signed addition 한 번만 수행한다. 수학적으로 같은 식 가운데 FPGA에서 carry와 MUX가 직렬로 놓이지 않는 표현을 선택한 것이다.

5.5 실패 3: Montgomery의 다섯 operand를 한 번에 더했다

초기 R16 Montgomery path는 선택된 네 shifted operand를 더해 $c_q T \bmod 2^9$ 를 만들고, 같은 조합 경로에서 $T[15:7]$ 까지 더해 $u[15:7]$ 을 완성했다.

```
1 shift_sum = op0 + op1 + op2 + op3;
2 u_hi      = shift_sum + T[15:7];
```

operand 폭은 9 bit로 작지만 다섯 항의 mode selection과 addition이 한 cycle에 모였다. 최종판은 C2에 $c_q T \bmod 512$ 만 등록하고, 등록된 $T[15:7]$ 은 다음 DSP-input cycle에서 더한다.

```
1 // C2 register
2 corr9_c2 <= op0 + op1 + op2 + op3;
3 t_c2     <= T;
4
5 // following DSP-input cycle
6 u_hi = t_c2[15:7] + corr9_c2;
```

```
7 u = {u_hi, t_c2[6:0]};
```

DSP1이 ML-DSA와 Montgomery mode에서 서로 다른 cycle 역할을 갖는다는 schedule을 이용했으므로 six-cycle latency와 $\Pi=1$ 을 유지했다. pipeline 최적화는 연산을 기계적으로 반으로 자르는 일이 아니라, 다음 hard block이 operand를 받는 시점까지 포함해 cut 위치를 정하는 일이다.

5.6 실패 4: 상수곱에 stage 하나를 더 넣자 기능이 깨졌다

NTRU+의 고정 상수곱은 처음에 shifted partial product 일곱 개와 $q = 4096 - 639$ 를 이용한 여러 fold를 두 cycle에 배치했다. adder tree를 balanced form으로 다시 써도 product와 modular fold의 carry depth 자체는 남았다. 이후 작은 input field를 table로 바꾸어 carry chain을 table output의 합과 한 번의 correction으로 줄였다. 최종 $1006x \bmod 3457$ 회로는 6-bit field 두 개의 table과 12-bit modular addition을 사용한다.

더 높은 주파수 실험에서는 이 상수곱을 세 stage로 늘려 보았다. 개별 block의 조합 depth는 줄지만 radix-3/삼항식 branch는 상수곱 결과와 별도의 difference rail이 같은 cycle에 도착한다고 가정했다. 상수곱만 한 cycle 늦어지자 NTRU+ INTT 결과가 어긋났다.

선택지는 두 가지였다.

1. 관련된 모든 branch와 metadata를 한 cycle 더 지연한다.
2. 두-cycle 계약을 유지하고 연산 표현을 바꾼다.

첫 선택은 공통 alignment FF와 전체 latency를 늘린다. 최종판은 두 번째를 택했다. raw subtraction의 underflow correction을 table 앞 12-bit adder로 수행하지 않고, underflow일 때 더해야 하는 상수 효과 $-1006 \times 639 \equiv 168 \pmod{3457}$ 를 table 선택에 미리 반영했다. 그 결과 correction carry chain 하나를 table data selection으로 바꾸면서 두-cycle arrival contract를 보존했다.

이 실패는 “register를 넣으면 timing은 좋아진다”는 문장이 불완전함을 보여 준다. multi-branch pipeline에서는 값, valid, mode, destination tag와 parallel operand가 같은 transaction으로 다시 만나는 cycle까지 함께 바뀐다.

5.7 실패 5: 연산기를 최대한 공유했더니 더 커졌다

한 실험판은 네 개의 runtime-generic folded ALU에 23-bit와 12-bit 연산을 모두 모으고, raw/correction two-pass register와 공통 delay rail을 두었다. 연산기 개수만 세면 매력적이지만 실제 FPGA 비용은 다음 위치로 이동했다.

- generic ALU 앞의 wide operand crossbar
- carry chain 뒤의 mode-dependent result MUX
- 여러 stage와 mode로 퍼지는 select fanout
- latency가 다른 branch를 끝까지 끌고 가는 alignment FF

NTRU+ lazy reduction도 다섯 internal correction을 제거하는 대신 intermediate가 13–14 bit로 넓어졌다. multiplier와 constant-multiplier 입력이 커지고 output canonicalization이 다시 필요했다. 최종 output에서 제거했던 adder/MUX가 돌아오므로 해당 branch의 순 area 이득이 사실상 사라졌다.

최종 구조는 큰 자원과 시점이 정확히 맞는 연산을 공유한다. 두 DSP, DSP register, final correction addition은 공유한다. NTRU+의 cycle-local add/sub는 12 bit로 남긴다. 이 경험에서 나온 기준은 다음과 같다.

공유 여부를 결정하는 기준

연산기 개수만 비교하지 않는다. 공유 전후의 operand MUX 폭, result MUX 위치, mode fanout, alignment register와 placement 거리까지 합쳐서 비교한다.

5.8 implementation directive만으로 150 MHz가 되지 않았다

중간 RTL을 Zynq-7020에 standalone OOC로 구현했을 때 default flow의 150 MHz WNS는 -0.374 ns였다. synthesis retiming과 timing-oriented place/route directive를 적용하면 -0.152 ns까지 회복했지만 여전히 timing은 달하지 않았다. 같은 flow에서 140 MHz는 $+0.026$ ns였다.

이 수치는 directive가 쓸모없다는 뜻이 아니다. RTL의 register, MUX, fanout 구조를 고친 뒤 implementation flow가 나머지 placement/routing margin을 회복한다. 최종 150 MHz 결과는 두 작업을 함께 적용한 결과다.

300 MHz 실험에서는 3.333 ns constraint에서 synthesis WNS -7.170 ns와 RAMB36E1 pulse-width slack -0.551 ns가 관찰되었다. 이 implementation point는 route를 진행하지 않았다. pulse-width violation은 조합 path에 register를 더 넣는 것으로 해결되지 않기 때문이다. 목표 주파수를 올리는 실험도 **RTL로 고칠 path, clock/device primitive 제한, 기능 정렬 비용**을 구분해 중단 기준을 세워야 한다.

실무 체크리스트

- 실패한 run도 requirement, WNS, source, destination, logic levels, logic/route 비율을 남긴다.
- 한 cycle 식에서 carry chain이 몇 번 이어지는지 회로로 다시 그린다.
- 빈 alignment stage를 먼저 찾고 새 stage 추가는 그다음에 검토한다.
- 한 branch의 latency를 바꾸면 parallel data와 metadata의 합류 cycle을 testbench에서 확인한다.
- 공유 실험은 ALU 수뿐 아니라 MUX, fanout, FF와 route까지 비교한다.
- directive로 얻은 margin과 RTL 구조 변경의 효과를 같은 것으로 쓰지 않는다.

6. Area와 inference 실제 사례

6.1 사례 1: BRAM inference가 먼저다

초기 synthesis는 true-dual-port memory의 두 write port를 합성기가 인식하는 형태로 표현하지 못해 RAM을 register로 풀었다. 네 bank hierarchy가 각각 약 27–29k LUT와 47,150 FF를 사용했고, 전체는 device 용량을 훨씬 초과했다. 이때의 우선순위는 address 폭 축소나 mux 미세 최적화가 아니라 inference template 수정이다.

최종 MDL_XXX_bram_dp.v는 port별 synchronous process를 분리한다.

```
1 (* ram_style = "block" *) reg [DATA_W-1:0] mem [0:DEPTH-1];
2
3 always @(posedge iSYS_CLK) begin : PORT_A
4     if (iENA) begin
5         if (iWEA) mem[iADDR_A] <= iDINA;
6         oDOUT_A <= mem[iADDR_A];
7     end
8 end
9
10 always @(posedge iSYS_CLK) begin : PORT_B
11     if (iENB) begin
12         if (iWEB) mem[iADDR_B] <= iDINB;
13         oDOUT_B <= mem[iADDR_B];
14     end
15 end
```

ram_style attribute만 붙이는 것으로 부족하다. port semantics가 primitive가 지원하는 동작과 맞아야 한다. read-first 동작과 same-row dual write가 없다는 scheduler 제약도 simulation assertion으로 남긴다.

6.2 사례 2: logical depth와 physical depth를 분리한다

최초 coefficient bank parameter는 11-bit address, depth 2048이었다. 최대 transform 길이는 1152이지만 네 physical bank가 각각 logical address 전체를 받을 수 있게 작성되어 있었다. 2048×23 true-dual-port bank 하나는 7-series에서 RAMB36과 RAMB18의 조합으로 구현되었고, 네 bank가 6 BRAM-tile equivalents를 소비했다.

최종 mapping은 logical address x 를 side와 row로 나눈다.

$$row = x[10 : 1], \quad side = x_0 \oplus x_k.$$

지원 transform에서 최대 row는 575이므로 각 bank는 576×23이면 충분하다. 이 크기는 RAMB36E1 하나에 들어가며 네 coefficient bank는 총 4 RAMB36이 된다.

6.2.1 왜 정보가 사라지지 않는가

row 에는 x_k 가 남아 있다($k \geq 1$). 그러므로

$$x_0 = side \oplus x_k$$

로 제거한 bit 0을 복원할 수 있다. 즉 ($side, row$)는 logical address와 일대일 대응한다. 단순히 address LSB를 버린 것이 아니라, 그 정보를 $side$ 에 보존한 compact mapping이다.

```
1 assign oSIDE      = addr[0] ^ addr[k];
2 assign oLOCAL_ADDR = addr[ADDR_W-1:1];
```

실제 RTL은 variable k 대신 4, 6, 7의 고정 tap을 case로 선택한다. mapping별 bijection, row bound, radix-2 bank balance를 exhaustive test로 확인한다.

6.3 사례 3: scheme별 ROM을 word packing으로 통합한다

최초 zeta_rom은 ML-KEM, ML-DSA, HAETAE, NTRU+768, NTRU+864/1152 ROM을 모두 instantiate한 뒤 mode mux로 출력을 선택했다. 합성 결과의 twiddle storage는 RAMB18 세 개, 즉 1.5 tile이었다.

최종판은 2048×32의 packed image 하나를 두 synchronous read port로 읽는다. 저장 구간은 다음과 같다.

Mode	Base word	Words	Packing
ML-DSA	0	256	23-bit value 1개/word
HAETAE	256	128	16-bit value 2개/word
ML-KEM	384	64	12-bit value 2개/word
NTRU+768	448	96	12-bit value 2개/word
NTRU+864/1152	544	144	12-bit value 2개/word
Falcon forward	688	512	16-bit value 2개/word
Falcon inverse	1200	512	16-bit value 2개/word

총 1712 word이므로 2048-word memory에 들어간다. narrow mode의 logical address LSB가 word 안의 low/high slot을 고른다.

```
1 phys_w = base_w + logical_addr[10:1];
2 slot_w = logical_addr[0];
3
4 out = slot_q ? word_q[31:16] : word_q[15:0];
```

packed ROM은 두 RAMB36을 사용한다. twiddle memory만 보면 1.5 tile에서 2 tile로 0.5 tile 증가했지만, Falcon forward/inverse table까지 포함한다. coefficient bank가 6 tile에서 4 tile로 줄어 전체 accelerator memory는 7.5 tile에서 6 tile로 1.5 tile 감소했다. 부분별 증감을 나눠 보아야 올바른 설명이 된다.

6.4 사례 4: pipeline metadata도 area다

data path만 세면 control storage를 놓치기 쉽다. 최초 core는 네 lane의 $va/vb/vc/vd$ 를 각각 12-stage pipeline으로 운반했고, BRAM read 뒤에 data/address/zeta를 다시 capture하는 register group도 있었다.

최종 transaction에서는 A/B/C lane이 항상 유효하고 radix-3일 때만 D가 없다. 따라서 tail valid는 layer-static butterfly mode로 계산한다.

```
1 wire ctl_is_r3 = (ctl_but_sel_q == BUT_R3_NTT) ||
```

```

2         (ctl_but_sel_q == BUT_R3_INTT);
3 wire wb_lane_d_valid = !ctl_is_r3;

```

최종 13-stage 기준으로 네 lane-valid shift register 52 FF를 제거한다. 이 숫자는 해당 register의 구조적 감소량이며, 전체 FF delta와 동일시하지 않는다. 다른 pipeline과 interface register도 동시에 바뀌었기 때문이다.

read data와 synchronous zeta output은 butterfly C1 register로 직접 들어가도록 정렬하여 duplicate capture stage도 제거했다. pipeline register를 무조건 줄인 것이 아니라, 같은 cycle boundary가 두 번 존재하는 부분을 찾은 것이다.

6.5 사례 5: 계측 logic을 production에서 생성하지 않는다

LOAD/CORE/DUMP/TOTAL cycle counter는 개발 중 유용하지만 accelerator protocol의 필수 기능은 아니다. 최종 top은 parameterized generate로 네 32-bit counter와 incrementer를 production configuration에서 제거한다.

```

1 generate
2   if (ENABLE_PERF_COUNTERS != 0) begin : GEN_PERF_COUNTERS
3     // four 32-bit cycle counters
4   end else begin : GEN_NO_PERF_COUNTERS
5     always @(posedge iSYS_CLK) begin
6       oLOAD_CYCLES  <= 0;
7       oCORE_CYCLES  <= 0;
8       oDUMP_CYCLES  <= 0;
9       oTOTAL_CYCLES <= 0;
10    end
11  end
12 endgenerate

```

하위 AXI load/dump beat counter도 같은 parameter를 사용한다. 검증용 관측성을 유지하면서 release hardware에 incrementer와 fanout을 남기지 않는 방법이다.

6.6 Area 감소를 설명하는 올바른 문장

최초 standalone과 최종 standalone의 역사적 결과는 LUT 5,278에서 4,736, FF 3,072에서 2,792, BRAM tile 7.5에서 6.0, DSP 4에서 4다. 각각 10.3%, 9.1%, 20.0% 감소와 DSP 동일이다. 그러나 package, top boundary, clock가 완전히 일치하는 단일 변수 실험은 아니다.

따라서 다음과 같이 쓴다.

권장 해석

최종 implementation point는 최초 standalone archive보다 542 LUT, 280 FF, 1.5 BRAM tile을 적게 사용하고 같은 네 DSP를 사용한다. compact coefficient mapping은 bank당 RAMB18을 제거했고, 다른 LUT/FF 변화에는 arithmetic, scheduler, metadata, wrapper 차이가 함께 포함된다.

특정 RTL 변경 하나가 전체 542 LUT를 줄였다고 쓰려면 그 변경만 다른 matched run이 추가로 필요하다.

7. Verilog 연산자의 실제 자원 비용

7.1 연산자 하나를 같은 조건에서 합성해 본다

Verilog의 짧은 기호와 FPGA primitive 수는 일대일로 대응하지 않는다. 이를 확인하기 위해 23-bit datapath에서 자주 쓰는 연산을 `operator_cost_all` top 하나에 각각 독립 instance로 배치하고 한 번의 OOC synthesis를 수행했다.

- Vivado 2020.2, xc7z020clg484-1
- `flatten_hierarchy none`
- 각 instance의 input/output 사이에 한 register boundary
- instance 사이의 함수와 제거를 막기 위한 hierarchy 보존
- place/route 이전의 post-synthesis primitive count

RTL 연산	LUT	CARRY4	FF	DSP	해석
23-bit <code>a+b</code>	23	6	69	0	한 carry chain
23-bit <code>a-b</code>	23	6	69	0	한 carry chain
23-bit <code>a==b</code>	8	2	47	0	bit 비교와 reduction
23-bit unsigned <code>a<b</code>	24	3	47	0	magnitude compare
<code>a>=8380417</code>	6	0	24	0	constant bit pattern으로 축소
23-bit 2:1 MUX	23	0	70	0	output bit당 LUT 하나
23-bit 4:1 MUX	23	0	117	0	4 data+2 select가 LUT6에 들어감
23-bit constant left shift	0	0	41	0	배선과 constant bit
23-bit variable left shift	64	0	51	0	multi-level barrel MUX
23×16 multiply	0	0	39	1	DSP48E1로 함수
23-bit modular subtraction	46	12	92	0	subtract와 conditional add
13-input reduction OR	3	0	14	0	작은 reduction tree

FF 수에는 연산기 자체뿐 아니라 실험 조건을 고정한 input/output boundary register가 포함된다. 예를 들어 23-bit add의 69 FF는 두 23-bit input register와 23-bit output register다. 그러므로 표의 FF 열을 순수 operator cost로 읽지 않는다. 반면 LUT, CARRY4와 DSP 열은 각 독립 instance의 조합 구조를 직접 보여 준다.

7.2 Addition과 subtraction: LUT가 carry를 구동한다

7-series의 CARRY4 하나는 네 bit의 carry propagation을 담당한다. 이 실험에서 23-bit addition과 subtraction은 각각

$$\left\lceil \frac{23}{4} \right\rceil = 6$$

개의 CARRY4와 23 LUT를 사용했다. LUT는 각 bit의 propagate/generate 또는 sum 선택을 만들고, CARRY4가 인접 bit의 carry를 빠르게 전달한다.

이 결과가 “23-bit adder는 항상 23 LUT”라는 법칙은 아니다. constant operand, 사용되지 않는 상위 carry, 주변 MUX와 downstream logic이 있으면 합성기가 LUT를 합치거나 제거한다. 그러나 RTL을 처음 검토할 때 23-bit carry 연산 하나는 대략 여섯 CARRY4 길이라고 보는 것은 유용하다.

7.2.1 왜 modular subtraction이 무거운가

한 cycle modular subtraction의 실험 코드는 raw difference와 conditional addition을 연속으로 둔다.

```
1 diff      = {1'b0,a} - {1'b0,b};
2 corrected = {1'b0,diff[22:0]} + {1'b0,q};
3 result    = diff[23] ? corrected[22:0] : diff[22:0];
```

합성 결과는 46 LUT와 12 CARRY4였다. 일반 subtract의 정확히 두 배인 carry resource가 한 register 경계에 놓인 셈이다. 150 MHz 실패 path에서 관찰한 11 CARRY4도 이 구조와 같은 직관을 준다. 최종 RTL이 early difference를 raw subtract와 conditional correction의 두 cycle로 나눈 이유를 area와 timing 양쪽에서 설명할 수 있다.

7.3 Comparator: 같은 23 bit라도 비용이 다르다

==는 bit별 equality와 reduction으로 구현할 수 있다. 이 실험의 23-bit equality는 8 LUT와 2 CARRY4였다. unsigned magnitude compare는 bit의 우선순위를 판단해야 하므로 24 LUT와 3 CARRY4를 사용했다.

constant compare는 더 작아질 수 있다. ML-DSA modulus 8,380,417과의 23-bit >= 비교는 6 LUT, CARRY4 0개로 줄었다. constant의 0/1 pattern과 reachable range를 합성기가 이용했기 때문이다.

따라서 다음 두 RTL은 폭이 같아도 같은 면적으로 예상하지 않는다.

```
1 wire ge_runtime = (value >= runtime_q);
2 wire ge_mldsa   = (value >= 23'd8380417);
```

현재 NTT RTL이 외부의 unconstrained PRM_Q를 넓은 arithmetic cone에 계속 전달하지 않고 scheme별 internal constant를 선택하는 이유도 여기에 있다. constant propagation은 comparator뿐 아니라 adder의 고정 bit와 MUX input도 줄일 수 있다.

7.4 MUX: input 개수와 bit 폭을 함께 센다

23-bit 2:1 MUX와 4:1 MUX는 이 isolated synthesis에서 모두 23 LUT였다. 한 output bit를 기준으로 2:1 MUX는 두 data와 select의 3-input function이고, 4:1 MUX는 네 data와 두 select의 6-input function이다. 둘 다 LUT6 하나에 들어간다.

이 결과에서 4:1 MUX가 항상 2:1과 같은 timing이라는 결론은 나오지 않는다. LUT input pin delay, select fanout와 placement가 다르다. 8:1 이상이 되거나 MUX 앞뒤에 arithmetic이 붙으면 LUT level 또는 dedicated MUX resource가 더 필요할 수 있다.

특히 다음 두 표현은 기능이 같아 보여도 합성 문맥이 다르다.

```

1 // Multiple arithmetic branches may become parallel operators plus a result MUX.
2 y = mode0 ? add_mod(a,b) :
3   mode1 ? add_mod(c,d) : add_mod(e,f);
4
5 // Select operands first, then use one physical arithmetic position.
6 case (mode)
7   0: begin op_a=a; op_b=b; end
8   1: begin op_a=c; op_b=d; end
9   default: begin op_a=e; op_b=f; end
10 endcase
11 y = add_mod(op_a,op_b);

```

최종 butterfly는 두 번째 구조를 사용한다. mode branch마다 function을 호출해 여러 adder와 result MUX를 만들지 않고, physical position의 operand를 먼저 선택한 뒤 arithmetic function을 한 번 호출한다.

7.5 Shift: constant와 variable은 전혀 다른 회로다

같은 « 연산자라도 shift amount가 compile-time constant인지 runtime signal인지가 중요하다. 23-bit $a \ll 5$ 는 LUT를 하나도 쓰지 않았다. bit 위치를 바꾸는 배선과 0 constant로 충분하기 때문이다.

반면 5-bit runtime shift amount를 사용한 23-bit left shift는 64 LUT였다. 여러 shift 후보를 단계별로 고르는 barrel MUX가 필요하다. NTT address와 modular arithmetic에서 고정 shift를 적극적으로 사용하는 이유는 곱셈을 덧셈으로 바꾸는 것뿐 아니라 variable barrel network를 피하기 위해서다.

7.6 Multiplication과 reduction

23x16 multiply에는 use_dsp를 지정했고 결과는 LUT 0, DSP48E1 한 개였다. operand가 DSP48E1 multiplier port 범위에 들어가므로 product와 output register가 hard block에 흡수된다. 최종 unifiedMUL은 이런 DSP를 두 개 사용하고, 두 instance의 합계가 네 DSP다.

constant multiplication은 항상 DSP를 써야 하는 것은 아니다. 최종 $1006x \bmod 3457$ block은 두 6-bit table과 addition/correction으로 구현되어 standalone post-route hierarchy에서 84 LUT와 25 FF, DSP 0개를 사용한다. 세 lane의 modular division by 3 block은 shift와 addition으로 구현되어 465 LUT와 216 FF, DSP 0개다. hard DSP를 네 개로 제한하는 대신 fabric area를 사용한 명시적인 trade-off다.

13-input reduction OR는 isolated synthesis에서 3 LUT였다. 수만 보면 작지만 그 결과가 BUSY, enable과 memory phase MUX를 동시에 구동하면 fanout과 route가 critical path가 될 수 있다. 최종 core가 |valid_pipe를 occupancy counter로 바꾼 이유는 LUT 세 개 자체보다 그 reduction 결과의 timing cone을 없애기 위해서다.

7.7 면적을 예상하는 순서

1. 각 cycle 경계 사이의 add/sub carry chain 수와 bit 폭을 센다.
2. arithmetic 앞 operand MUX와 뒤 result MUX 가운데 어느 쪽이 생기는지 그린다.
3. variable shift, runtime modulus, generic multiply처럼 작은 기호가 큰 network를 만드는 표현을 표시한다.
4. pipeline data뿐 아니라 valid, mode, address와 destination tag의 register bit도 센다.
5. memory는 선언 bit 수가 아니라 target primitive의 width/depth configuration으로 올림해 센다.

6. 마지막에는 반드시 post-synthesis hierarchy와 primitive count로 예상이 맞는지 확인한다.

이 프로젝트의 최종 standalone hierarchy에서 complete unifiedBUT은 2,704 LUT, 2,038 FF와 네 DSP를 사용한다. 하위 두 unifiedMUL은 같은 RTL이지만 각각 459/378 LUT로 다르게 보였다. downstream trimming과 hierarchy 간 LUT combining 때문이다. 하위 block 수를 단순 합산하거나 동일 RTL의 한 instance 수치를 절대값으로 일반화하지 않는다.

실무 체크리스트

- add/sub는 먼저 $\lceil W/4 \rceil$ CARRY4 chain을 예상한다.
- conditional correction은 두 번째 carry chain과 MUX가 붙는지 확인한다.
- 4:1 bit MUX는 LUT6 하나에 들어가도 select fanout은 별도로 본다.
- constant shift는 배선, variable shift는 barrel MUX로 생각한다.
- multiplication은 DSP inference report와 내부 register 흡수를 확인한다.
- 작은 OOC 결과와 complete-system post-route 결과를 같은 수치로 취급하지 않는다.

재현용 RTL, Tcl과 실제 summary는 repository의 examples/operator_cost에 있다. 하나의 in-memory project에서 synthesis를 한 번만 실행하므로 Vivado project directory나 checkpoint를 저장소에 남기지 않는다.

8. 공유 산술기와 pipeline 설계

8.1 설계 목표를 연산식과 자원으로 동시에 적는다

최종 unifiedMUL의 목표는 두 종류의 modular multiplication을 같은 두 DSP48E1과 arithmetic/register path에서 실행하는 것이다.

- ML-DSA: $q = 8,380,417$, 23-bit operand path
- Narrow primes: R16 Montgomery multiplication, $R = 2^{16}$
- Narrow modulus: ML-KEM 3,329, HAETAE 64,513, NTRU+ 3,457, Falcon 12,289
- Latency: six pipeline stages
- Throughput: mode가 유지되는 동안 매 cycle 새 입력 수락, II=1

하나의 unifiedBUT은 unifiedMUL 두 개를 사용하므로 전체 butterfly arithmetic은 DSP48E1 네 개를 사용한다.

8.2 DSP0: 공통 low product

두 path 모두 먼저 A 와 B 의 low 16-bit product를 필요로 한다.

```
1 (* use_dsp = "yes" *) wire [38:0] c1_mul_lo_w;  
2 assign c1_mul_lo_w = iA[22:0] * iB[15:0];  
3  
4 always @(posedge iSYS_CLK)  
5   if (iFSM_START) c1_mul_lo_r <= c1_mul_lo_w;
```

ML-DSA에서는 23-bit B 를 low 16-bit와 high 7-bit로 분할한다.

$$AB = AB_{lo} + 2^{16}AB_{hi}.$$

DSP0의 low product와 DSP1의 high partial product를 정렬해 더한다. Narrow-prime mode에서는 DSP0 output의 low 32 bit가 $T = AB$ 다.

8.3 Narrow-prime path: R16 Montgomery multiplication

Narrow mode는 다음 연산을 수행한다.

$$T = AB, \quad u = Tq^{-1} \bmod 2^{16}, \quad t = \frac{T - uq}{2^{16}}.$$

$t < 0$ 이면 q 를 한 번 더해 canonical result를 만든다. 네 modulus는 모두 $q \equiv 1 \pmod{2^7}$ 을 만족하므로

$$q^{-1} = 1 + 2^7 c_q \pmod{2^{16}}$$

로 쓸 수 있다. 따라서 u 의 low 7 bit는 T 의 low 7 bit와 같고, high 9 bit는 선택된 shift와 9-bit addition으로 계산할 수 있다.

실제 RTL은 mode별로 필요한 shifted value를 고른 뒤 하나의 9-bit 합에 넣는다.

```

1 case (mode)
2   CTL_KYBER: begin op0 = -m3; op1 = m5; op2 = m6; end
3   CTL_HAETAETAE: begin op0 = m4; end
4   CTL_NTRU: begin op0 = m2; op1 = m3;
5               op2 = m5; op3 = m7; end
6   CTL_FALCON: begin op0 = -m2; op1 = -m3; end
7 endcase
8 corr9 = op0 + op1 + op2 + op3;

```

여기서 $m_2, \dots, m_7$ 은 T 의 일부 bit를 고정 shift한 9-bit 값이다. 9-bit wraparound가 modulo 512 연산을 제공한다. 네 modulus마다 넓은 constant multiplier를 병렬로 만든 뒤 고르는 구조를 피하고, mode select와 작은 addition을 공유한다.

8.4 DSP1: mode별 곱과 정렬 산술 공유

두 번째 DSP는 mode에 따라 다른 역할을 한다.

Mode	DSP1 multiplication	DSP ALU/PREG 동작
ML-DSA	$A \times B_{hi}$	DSP0 low product의 정렬된 high 부분과 덧셈
Narrow prime	$u \times q$	정렬된 T 에서 uq 를 뺌

최종 synthesis branch는 DSP48E1의 MREG, ALU, PREG를 사용한다. ML-DSA와 Montgomery path의 fabric mux를 multiplier 바로 앞에 깊게 두지 않고 DSP의 A/D/C 입력과 ALUMODE/INMODE로 역할을 바꾼다. simulation branch는 같은 latency의 behavioral multiplication과 register를 사용하므로, 두 branch가 bit-accurate하고 cycle-accurate한지 regression에서 함께 확인해야 한다.

8.5 공유 register는 live bit를 기준으로 설계한다

각 stage의 payload 폭은 두 mode의 필요한 정보를 담을 만큼만 잡는다.

Stage	ML-DSA에서 보존할 값	Narrow mode에서 보존할 값
C1	23×16 low product	$T = AB$
C2	low product 일부	T 와 $c_q T \pmod{512}$
C3	quotient 준비용 high/low 조각	DSP1 subtraction용 정렬된 T
C4	residual과 borrow 정보	$T - uq$ 는 DSP1 PREG에 유지
C5	base-minus-borrow 결과	negative result에 조건부 $+q$
C6	ML-DSA canonical correction	C5 narrow result 출력

두 mode가 같은 cycle에 동시에 실행되지 않으므로 payload의 live range가 겹치지 않는 bit는 공유할 수 있다. 다만 mode mux가 wide payload 전체에 생기지 않도록 양쪽에서 실제로 소비되는 low part만 선택하고, 한 mode에서 의미 없는 upper part는 다른 mode 값을 그대로 연결한다.

8.6 최종 correction addition 공유

C5에서는 ML-DSA의 base-minus-borrow와 narrow Montgomery result의 조건부 $+q$ 를 하나의 signed 18-bit adder로 표현한다.

```

1 shared_a = is_dili ? sign_extend(dili_base)
2             : sign_extend(mont_quotient);
3 shared_b = is_dili ? -borrow
4             : (mont_negative ? mont_q : 0);
5 shared_sum = shared_a + shared_b;

```

공유 가능성은 “둘 다 덧셈처럼 보인다”에서 나오지 않는다. 두 연산이 같은 pipeline stage에 있고, mode가 transaction 동안 고정되고, 필요한 signed range가 18 bit 안에 들어간다는 세 조건에서 나온다.

8.7 ML-DSA path의 폭을 줄이는 방법

ML-DSA modulus는

$$2^{23} \equiv 2^{13} - 1 \pmod{q}$$

관계를 갖는다. 최종 RTL은 23-bit operand split으로 얻은 product 조각을 13-bit radix 위치에 맞춰 재배열하고, 필요한 low subtraction과 high signed addition을 나눠 계산한다. 이는 알려진 modulus relation을 shared datapath에 맞게 배치한 것이며 새로운 reduction algorithm을 주장하는 근거가 아니다.

중요한 RTL 습관은 중간값의 증명된 범위를 사용해 폭을 정하고 assertion으로 남기는 것이다. 예를 들어 signed high base와 final canonical range가 예상 범위를 벗어나면 simulation에서 즉시 error를 발생시킨다. 이런 assertion은 합성에서 제거되므로 production area에는 포함되지 않는다.

8.8 산술 timing의 현재 위치

최종 standalone 150 MHz 최악 경로는 mode register에서 butterfly modular-add register까지이며, 7 logic levels와 6.522 ns data delay를 보였다. complete BRAM system 최악 경로도 butterfly input에서 constant-multiply register까지 10 logic levels, 6.550 ns다. memory-address path가 정리된 뒤 arithmetic이 다시 timing closure의 중심이 된 것이다.

다음 개선을 시도한다면 mode fanout, constant-add partition, stage boundary를 matched OOC run으로 비교해야 한다. 단순히 DSP를 더 쓰거나 pipeline latency를 늘리기 전에 $\Pi=1$, 네-DSP limit, layer cycle 계약을 함께 보존해야 한다.

실무 체크리스트

- mode별 수식을 먼저 쓰고 같은 cycle의 공통 연산을 찾는다.
- operand와 중간값의 signed range를 증명해 register 폭을 정한다.
- DSP 내부 MREG/PREG와 fabric register의 경계를 따로 센다.
- mode mux가 DSP/carry 앞 critical path에 붙는지 report로 확인한다.
- latency와 initiation interval을 별도 지표로 검증한다.

9. Scheduler, memory mapping, interface

9.1 연산량보다 빈 cycle을 먼저 센다

같은 butterfly 수라도 layer 시작, BRAM read, pipeline fill/drain, 다음 layer descriptor 준비 때문에 cycle 수가 달라진다. 최종 radix-2 schedule의 관측식은 다음처럼 정리된다.

$$C_{core} = L(N_{issue/layer} + 15).$$

ML-KEM은 7개 layer와 layer당 64 issue로 $7(64 + 15) = 553$ cycle, ML-DSA/HAETAE는 8개 layer로 $8(64 + 15) = 632$ cycle이다. 이 식의 15는 단일 butterfly latency만 뜻하지 않는다. address issue부터 final writeback과 layer 전환까지의 관측 overhead다.

최종 FSM은 다음 layer descriptor를 미리 계산하고, 마지막 writeback edge의 lookahead로 다음 layer를 시작한다. address generator도 first transaction seed를 descriptor에 포함해 layer 시작 뒤의 별도 준비 cycle을 줄인다.

9.2 NTRU+864의 half-empty issue 제거

NTRU+864의 특수 step-3 구간을 일반 schedule로 표현하면 일부 transaction에서 lane을 충분히 사용하지 못한다. 최종 ntt_addrngen은 ADDR_MODE_N864_STEP3를 별도로 두어 유효 coefficient 조합을 직접 발행한다. 이는 memory layout 자체를 바꾸는 mode가 아니다. logical address schedule만 바꾸고 affine mapping은 유지한다.

이 구분 덕분에 scheduler test는 issue address/valid를 검사하고, mapper test는 bijection/bank conflict를 독립적으로 검사할 수 있다. schedule와 physical layout을 한 always block에 섞으면 어느 쪽의 오류인지 분리하기 어렵다.

9.3 Core-cycle 결과

Configuration	최초 cycle	최종 cycle	Cycle 감소	100/150 MHz 시간 감소
ML-KEM-256	557	553	0.7%	33.8%
ML-DSA-256	636	632	0.6%	33.8%
HAETAE-256	636	632	0.6%	33.8%
NTRU+768	1,709	1,513	11.5%	41.0%
NTRU+864	2,053	1,761	14.2%	42.8%
NTRU+1152	2,605	2,313	11.2%	40.8%

마지막 열은 최초 100 MHz와 최종 150 MHz implementation point를 사용한 core-only 시간이다. radix-2-only mode는 cycle 감소가 작고 주파수 상승 효과가 중심이다. Mixed-radix NTRU+는 scheduler cycle 감소가 추가로 기여한다.

9.4 AXI LOAD: beat 안의 병렬성을 memory lane으로 전달한다

64-bit AXI input 한 beat는 ML-DISA에서 32-bit coefficient 두 개, narrow mode에서 16-bit coefficient 네 개를 담는다. 최종 loader는 accepted beat마다 최대 네 logical write를 같은 cycle에 만든다.

```
1 oLOAD_WE  <= 1'b1; oLOAD_ADDR  <= coeff_q;
2 oLOAD1_WE <= 1'b1; oLOAD1_ADDR <= coeff_q + 1;
3 oLOAD2_WE <= !mldsa;oLOAD2_ADDR <= coeff_q + 2;
4 oLOAD3_WE <= !mldsa;oLOAD3_ADDR <= coeff_q + 3;
```

지원하는 모든 N 이 packing factor로 정확히 나누어지므로 각 lane의 upper-bound comparator를 반복할 필요가 없다. 한 번의 all_issued/final_accept 판정으로 충분하다. 불변조건을 이용해 세 개의 11-bit 비교기를 제거한 사례다.

oWS_TREADY는 receive state에서 항상 1이고 내부 bubble을 삽입하지 않는다. 따라서 source가 TVALID를 연속으로 주면 매 cycle 한 beat를 받는다.

9.5 AXI DUMP: backpressure보다 먼저 reservation한다

BRAM request는 발행 후 취소할 수 없다. downstream TREADY가 내려간 뒤 FIFO가 꽉 차면 이미 날아오는 response를 버리게 된다. 최종 dump는 현재 FIFO count만 보지 않고, 아직 도착하지 않은 request까지 slot을 예약한다.

```
1 wire pop      = (count_q != 0) && iWM_TREADY;
2 wire can_issue = (credit_q != 0) || pop;
3 wire issue     = active_q && !all_issued && can_issue;
4
5 case ({issue,pop})
6   2'b10: credit_q <= credit_q - 1'b1;
7   2'b01: credit_q <= credit_q + 1'b1;
8 endcase
```

같은 cycle에 pop이 있으면 그 slot을 새 request에 즉시 재사용한다. 정상 TREADY=1 상태에서 refill bubble 없이 $II=1$ 을 유지하면서, 임의의 backpressure에서는 overflow를 막는다. FIFO payload는 reset하지 않는다. count_q=0이 모든 오래된 entry를 invalid로 만들기 때문에 payload reset을 없애 LUTRAM inference와 packing을 돕는다.

9.6 Interface 두 개를 비교하는 법

AXI4-Stream과 AXI BRAM Controller는 같은 engine을 사용하지만 PS-PL data movement가 다르다. 따라서 비교에는 두 층이 있다.

- **Core-only**: 같은 engine의 cycle와 standalone area
- **End-to-end**: configuration, cache handling, DMA/MMIO transfer, polling, output availability를 포함한 board 시간

최종 complete-system에서 stream 쪽은 AXI DMA MM2S/S2MM payload FIFO를 포함한다. MM2S FIFO는 68 bit, S2MM FIFO는 72 bit라서 각각 RAMB36 하나와 남은 2 bit용 RAMB18 하나를 사용한다. 따라서 accelerator의 6 RAMB36에

$$2 \times \text{RAMB36} + 2 \times 0.5 \text{ RAMB18} = 3 \text{ tiles}$$

가 더해져 9 tile이 된다. 이것은 AXI4-Stream protocol 자체나 NTT arithmetic의 비용이 아니라 해당 DMA FIFO configuration의 data-movement 비용이다.

AXI BRAM Controller configuration은 이 payload FIFO 없이 coefficient memory에 접근하며, 현재 post-route 결과에서 controller 자체는 BRAM을 사용하지 않는다. complete system의 area 차이를 core arithmetic 개선으로 해석하면 안 된다.

실무 체크리스트

- layer당 issue, fill, drain, transition cycle을 분리해 쉰다.
- logical schedule와 physical mapping을 독립 module/test로 둔다.
- ready/valid throughput과 total latency를 별도로 측정한다.
- in-flight response까지 FIFO credit에 포함한다.
- interface 비교는 같은 device, clock, tool, implementation directive로 한다.

10. 최적화를 깨지 않게 검증하는 방법

10.1 검증 대상은 값, cycle, protocol 세 층이다

수학 결과만 같다고 hardware optimization이 끝난 것은 아니다. pipeline을 바꾼 뒤에는 다음 세 층을 따로 확인한다.

층	확인할 계약	이 사례의 예
Value	modular result와 transform coefficient	multiplier/butterfly/full NTT reference 비교
Cycle	input, result, valid, last, address tag 정렬	issue-to-writeback latency, layer done lookahead
Protocol	ready/valid, backpressure, reset, phase exclusivity	AXI gap/TREADY stall/TLAST, LOAD-CORE-DUMP

10.2 latency를 하나의 상수로 관리한다

최종 core는 XXX_ISSUE_TO_WB_LAT=13을 definitions file에 한 번만 정의한다. top과 core가 서로 다른 숫자를 localparam으로 가지면 arithmetic latency 수정 후 metadata pipeline이 조용히 어긋날 수 있다.

testbench에서는 다음 사건의 cycle 차이를 검사한다.

1. address generator의 issue valid
2. BRAM read request와 response
3. butterfly input acceptance
4. writeback valid/address/data
5. final writeback과 layer/core done

valid만 맞추면 충분하지 않다. A/B/C/D address tag와 두 zeta, radix-3의 D-lane absence, inverse mode의 twiddle 선택이 같은 transaction을 가리켜야 한다.

10.3 assertion으로 optimization 전제를 코드에 남긴다

최종 RTL의 simulation-only check는 설계자가 area/timing을 줄일 때 사용한 전제를 검사한다.

- transform 실행 중 crypto, N , inverse parameter가 바뀌지 않는다.
- physical row가 576-entry bank 범위를 넘지 않는다.
- 같은 BRAM row에 두 port가 동시에 write하지 않는다.

- multiplier operand와 intermediate signed range가 증명 범위 안에 있다.
- Montgomery의 $T - uq$ low 16 bit가 0이고 최종 값이 $[0, q)$ 다.
- dump FIFO count와 credit이 depth를 넘지 않는다.

```

1 `ifndef SYNTHESIS
2 always @(posedge iSYS_CLK) begin
3     if (response && (count_q == FIFO_DEPTH) && !pop)
4         $error("unreserved BRAM response");
5     if (credit_q > FIFO_DEPTH)
6         $error("dump FIFO credit overflow");
7 end
8 `endif

```

assertion은 test vector가 우연히 문제 조건을 만들지 않아도 내부 계약 위반을 찾는다. SYNTHESIS guard로 production hardware에는 들어가지 않는다.

10.4 변경 종류별 필요한 검증

변경	최소 검증
Register 위치/latency 변경	cycle scoreboard, valid/last/address alignment, back-to-back transaction
Bit width/signed 변경	경계값, random arithmetic, range assertion
Memory mapping 변경	전 주소 bijection, row bound, bank/port conflict, roundtrip
BRAM coding template 변경	simulation read-during-write semantics와 synthesis inference report
FIFO/credit 변경	random source gap와 destination backpressure, overflow/duplicate/loss check
Mode sharing 변경	mode별 unit test와 mode-stability assertion
Scheduler 변경	issue sequence, per-layer count, all transform roundtrip/reference

10.5 실행 가능한 regression

Companion AXI4-Stream 저장소 root에서 최종 RTL regression을 실행한다.

```
./tool_sim/run_all.sh
```

성공 시 다음 다섯 test가 각각 PASS로 표시된다.

1. memory mapping: bijection과 radix-2 bank balance
2. unified modular multiplier: 30 arithmetic cases
3. unified butterfly: 14 radix-2/radix-3/trinomial cases
4. full NTT/INTT: 지원하는 8 configuration
5. AXI integration: source gap, backpressure, TLAST

마지막 줄은 ALL_AXI_STREAM_TESTBENCHES_PASS다. 보존한 100 MHz baseline은 다음처럼 실행한다.

```
./old/tool_sim/run_all.sh
```

이 regression은 unified butterfly, ML-KEM radix-2 roundtrip, NTRU+768의 trinomial/radix-3/radix-2 roundtrip을 검사하고 3/3 PASS를 출력한다. 두 suite는 변경 전후 모두 실행 가능하므로 “새 코드만 맞는다”가 아니라 보존 baseline의 동작도 재확인할 수 있다.

10.6 simulation PASS와 implementation PASS를 구분한다

Icarus regression은 기능과 protocol을 확인하지만 DSP48E1 placement, BRAM inference, timing closure를 증명하지 않는다. 최종 acceptance에는 다음이 모두 필요하다.

- RTL regression PASS
- synthesis에서 DSP 4개와 의도한 BRAM 확인
- post-route setup/hold failing endpoint 0
- bitstream 생성과 DRC error 0
- board KAT/reference 비교 PASS
- benchmark의 clock와 measurement interval 기록

이 사례의 packed-BRAM board binary는 benchmark 전에 6개 scheme의 NTT와 INTT, 총 12개 transform을 software reference와 비교했다. completed state가 기록된 뒤 성능을 측정했으므로 기능 검증과 timing measurement의 실행 순서가 분명하다.

11. 반복 가능한 최적화 workflow

11.1 한 iteration의 순서

1. 기준점 고정: commit, top, part, clock, directives를 기록한다.
2. 기능 baseline: unit/full/protocol regression을 실행한다.
3. 합성 구조 확인: BRAM/DSP/SRL inference와 hierarchy area를 본다.
4. post-route 경로 분류: memory, arithmetic, control, interface로 나눈다.
5. 가설 하나 선택: 예를 들어 “logical mapping이 RAM pin 앞에 있다”.
6. 최소 구조 변경: register boundary와 metadata contract를 함께 고친다.
7. 회귀: value/cycle/protocol test를 모두 실행한다.
8. 동일 조건 재구현: 새 top- N path와 area delta를 기록한다.
9. 유지 또는 폐기: 개선이 재현될 때만 main branch에 남긴다.

여러 optimization을 한 번에 넣으면 어떤 변경이 WNS를 움직였는지 알 수 없다. 다만 서로 분리할 수 없는 구조 변경, 예를 들어 address register와 valid alignment는 하나의 transaction으로 다뤄야 한다.

11.2 Vivado에서 남길 최소 report

```
report_utilization -hierarchical -file utilization_hier_synth.rpt
report_timing_summary -file timing_summary_synth.rpt

opt_design
place_design
phys_opt_design
route_design

report_utilization -hierarchical -file utilization_hier_post_route.rpt
report_timing_summary -file timing_summary_post_route.rpt
report_timing -delay_type max -max_paths 50 \
    -sort_by slack -file timing_paths_post_route.rpt
report_drc -file drc_post_route.rpt
```

Post-synthesis report는 inference와 조합 구조를 빠르게 보는 데 유용하다. 최종 area와 timing 주장은 routing을 포함한 post-route report를 기준으로 한다.

11.3 경로를 RTL 질문으로 번역한다

Report 관찰	RTL에서 묻는 질문
RAM address pin이 destination	address mapping, phase mux, enable decode가 한 cycle에 겹치는가?
Route 비율이 80% 이상	source/destination이 멀거나 fanout이 큰가? register boundary를 옮길 수 있는가?
Control net fanout이 큼	동일 조건을 여러 block에서 재계산하는가? phase 불변조건으로 없앨 수 있는가?
Carry chain이 여러 묶음 연속	compare/subtract/correct가 직렬인가? range로 correction 횟수를 제한할 수 있는가?
DSP 앞 LUT가 깊음	mode select 또는 sign/width 변환이 multiplier 입력에 붙었는가?
예상보다 FF가 많음	lane metadata, debug counter, FIFO payload reset이 필요한가?
BRAM이 0	port process, reset, read semantics가 inference template과 맞는가?

11.4 실험 기록 형식

각 변경은 다음 표 한 장으로 남기면 충분하다.

필드	기록 내용
Hypothesis	report와 RTL을 연결한 원인 문장
Changed files	실제 수정한 module과 parameter
Preserved contract	latency, II, memory semantics, supported mode
Regression	test 이름과 PASS/FAIL
Synthesis	LUT/FF/BRAM/DSP와 inference warning
Post-route	WNS/TNS/WHs, source/destination, logic/route 비율
Decision	keep/revert와 이유

“WNS가 좋아졌다”만 적으면 placement seed 변화와 구조 개선을 구분하기 어렵다. 문제 path가 사라졌는지, 동일 class의 endpoint 수가 줄었는지, 새 병목이 어디로 이동했는지를 함께 기록한다.

11.5 이 사례에서 폐기해야 했던 단순 해석

- **디렉터리 이름이 달성 주파수다**: 200mhz branch는 200 MHz 미충족이었다.
- **pipeline이면 무조건 빨라진다**: DUMP를 끊은 뒤 LOAD의 routing 병목이 남았다.
- **shift/add면 DSP보다 작다**: 여러 mode의 병렬 constant logic은 LUT/carry를 크게 쓸 수 있다.
- **BRAM attribute면 BRAM이 된다**: port semantics가 맞지 않으면 RAM이 FF로 풀릴 수 있다.
- **전체 BRAM 차이는 core 차이다**: stream의 추가 3 tile은 DMA FIFO였다.
- **FF 증가는 실패다**: complete system FF는 늘어도 standalone core는 줄고 timing은 닫힐 수 있다.

이 항목들은 독립적인 “나쁜 코드 장”으로 반복할 필요가 없다. 각 관찰이 나온 실제 path와 hierarchy에 붙여 두어야 언제 적용할 수 있는지 알 수 있다.

11.6 최종 review checklist

실무 체크리스트

- RTL과 문서의 parameter, latency, resource 수가 일치한다.
- 사용하지 않는 debug/performance logic은 synthesis에서 제거된다.
- memory depth/width와 실제 RAMB primitive 수가 설명된다.
- top-level별 area 범위가 표 제목에 들어간다.
- unmatched historical comparison에는 caveat가 붙는다.
- core cycle와 end-to-end board latency를 섞지 않는다.
- test 명령과 성공 marker가 저장소에서 바로 실행된다.
- generated report와 artifact에는 tool/version/clock 정보가 남는다.

12. 측정 결과와 설계 결론

12.1 최종 세 범위의 post-route 결과

모든 최종 결과는 Vivado 2020.2, xc7z020c1g484-1, 6.666 ns constraint를 사용한다. post-route utilization을 primary area result로 사용한다.

Scope	LUT	FF	R36	R18	Tile	DSP	WNS
AXI4-Stream + DMA + Zynq	9,014	8,764	8	2	9.0	4	+0.075
Packed AXI BRAM Controller + Zynq	8,908	6,825	6	0	6.0	4	+0.006
Standalone MDL OOC	4,736	2,792	6	0	6.0	4	+0.089

WNS 단위는 ns다. 세 구현 모두 setup/hold failing endpoint가 0이다. standalone WNS는 internal synchronous path를 나타내며 placed interface endpoint가 있는 complete-system timing을 대신하지 않는다.

12.2 재현을 위한 post-synthesis 결과

Scope	LUT	FF	R36	R18	DSP
AXI4-Stream + DMA + Zynq	9,415	9,055	8	2	4
Packed AXI BRAM Controller + Zynq	9,399	7,695	6	0	4
Standalone MDL OOC	4,762	2,792	6	0	4

합성 후와 배치배선 후 LUT 수가 다른 것은 physical optimization과 packing의 영향이다. 보고서 단계가 없는 숫자 하나만 남기면 나중에 차이를 재현하기 어렵다.

12.3 100 MHz baseline과의 역사적 변화

Version/scope	Device	MHz	LUT	FF	BRAM tile	DSP
최초 ntt_top OOC	clg400-1	100	5,278	3,072	7.5	4
최종 MDL OOC	clg484-1	150	4,736	2,792	6.0	4
최초 complete stream system	clg484-1	100	9,108	7,844	16.5	4
최종 complete stream system	clg484-1	150	9,014	8,764	9.0	4

Standalone 역사적 delta는 LUT 10.3%, FF 9.1%, BRAM tile 20.0% 감소이고 DSP는 같다. 그러나 top-level이 최초 MDL_XXX_ntt_top, 최종 MDL이며 package와 clock constraint도 다르다. 따라서 이 수치는 개발 결과의 변화량이지 matched-device one-variable experiment가 아니다.

Complete stream system에서는 LUT가 1.0% 줄고 FF가 11.7% 늘며 BRAM tile이 45.5% 줄었다. FF 증가는 final interface/timing configuration을 포함한다. BRAM 감소도 core memory organization과 generated IP configuration이 함께 들어간 결과이므로 전부 arithmetic core에 귀속하지 않는다.

12.4 자원별 area-time product

FPGA에서는 LUT, FF, BRAM과 DSP가 서로 대체 가능한 동일 면적 단위가 아니다. 따라서 임의의 가중치로 하나의 ATP를 만들지 않고 다음 세 값을 따로 계산한다.

$$ATP_{LUT} = N_{LUT} T, \quad ATP_{FF} = N_{FF} T, \quad ATP_{BRAM} = N_{tile} T.$$

여기서 $T = C_{core}/f_{clk}$ 이며 interface transfer를 제외한 core cycle과 standalone resource count를 사용한다.

Configuration	Core time 감소	LUT-time 감소	FF-time 감소	BRAM-time 감소
ML-KEM-256	33.8%	40.6%	39.8%	47.0%
ML-DSA-256	33.8%	40.6%	39.8%	47.0%
HAETAE-256	33.8%	40.6%	39.8%	47.0%
NTRU+768	41.0%	47.0%	46.4%	52.8%
NTRU+864	42.8%	48.7%	48.0%	54.3%
NTRU+1152	40.8%	46.9%	46.2%	52.6%

예를 들어 ML-KEM은 557 cycle/100 MHz의 5.570 us에서 553 cycle/150 MHz의 3.687 us로 줄었다. 여기에 LUT 감소 4736/5278을 함께 곱하면

$$1 - \frac{4736 \times 3.687}{5278 \times 5.570} = 40.6\%$$

의 LUT-time product 감소가 된다. NTRU+는 clock 상승에 scheduler cycle 감소까지 더해져 ATP 감소가 더 크다. DSP는 네 개로 동일하므로 DSP-time product의 감소율은 각 mode의 core-time 감소율과 같다.

이 역시 package, clock와 여러 RTL 변경이 함께 다른 역사적 비교다. ATP 표는 최초부터 최종까지의 전체 발전을 요약하며, 특정 adder나 memory mapping 하나의 독립 효과를 나타내지 않는다. complete interface ATP를 주장하려면 두 system의 동일 범위 end-to-end time과 complete-system resource를 별도로 사용해야 한다.

12.5 최종 end-to-end board latency

아래 시간은 accelerator configuration/start, cache handling, PS-PL input/output, completion polling, output availability를 포함한 1,000회 평균이다. Cortex-A9 counter는 666,666,687 Hz, PL은 150 MHz다.

Scheme	Stream NTT	Stream INTT	BRAM NTT	BRAM INTT
ML-DSA	11.932 us	11.932 us	34.055 us	34.052 us
ML-KEM	9.713 us	9.715 us	21.232 us	21.230 us
HAETAE	10.126 us	10.127 us	21.772 us	21.772 us
NTRU+768	19.832 us	19.834 us	54.587 us	54.587 us
NTRU+864	22.066 us	22.070 us	61.057 us	61.055 us

NTRU+1152	27.832 us	27.832 us	79.955 us	79.952 us
-----------	-----------	-----------	-----------	-----------

Falcon modular NTT/INTT mode는 RTL simulation으로 검증했으며 이 board benchmark에는 포함되지 않았다. 개별 transform의 BRAM path가 stream보다 느리다는 결과를 area와 바꾸어 말하면 안 된다. interface/data movement의 latency와 resource trade-off를 그대로 제시한다.

12.6 무엇이 실제로 개선을 만들었는가

1. **측정 가능성**: inference 실패를 PPA baseline에서 제외하고 정상 BRAM mapping부터 확보했다.
2. **memory 경계**: logical mapping과 phase mux를 registered physical command/tag로 바꾸어 RAM pin 앞 critical path를 끊었다.
3. **저장 밀도**: bijective side/row mapping으로 coefficient bank depth를 실제 최대치 576으로 줄였다.
4. **공유**: 두 DSP와 stage register를 ML-DSA/R16 path가 공유하고, 마지막 correction addition도 공유했다.
5. **불변조건 활용**: 고정 port pattern, lane validity, exact beat size, fixed mode를 이용해 runtime compare/control을 줄였다.
6. **처리율 보존**: 더 긴 pipeline에서도 $\Pi=1$ 을 유지하고 scheduler의 빈 cycle을 줄였다.
7. **범위 분리**: standalone core와 두 complete PS-PL system을 따로 보고해 interface cost를 arithmetic cost로 오해하지 않게 했다.

가장 일반적인 결론은 “pipeline을 많이 넣는다”가 아니다. report가 지목한 경로에 transaction boundary를 만들고, 그 경계를 따라 data와 metadata를 함께 옮기며, 실제 address range와 mode 불변조건으로 storage/control을 줄이는 것이 이 사례의 설계 방법이다.

A. 용어 사전

용어	이 책에서의 의미
Area	LUT, FF, BRAM, DSP 등 구현 자원. 반드시 top-level 범위와 report 단계를 함께 쓴다.
BRAM inference	RTL memory가 RAMB36/RAMB18 hard block으로 합성되는 과정
Canonical result	modular value가 정한 범위 $[0, q)$ 안에 있는 표현
Carry chain	FPGA slice 안에서 addition/subtraction carry를 빠르게 전달하는 전용 경로
Control set	register가 공유하는 clock, reset, enable 조합. packing에 영향을 준다.
Critical path	특정 timing check에서 slack이 가장 작은 path
Data path delay	source에서 destination까지 cell logic와 routed net delay의 합
DSP48E1	7-series의 multiplier, pre-adder, ALU, pipeline register를 포함 hard block
Fanout	한 net이 구동하는 load 수. 높은 fanout은 넓은 routing을 요구할 수 있다.
Initiation interval (II)	연속 transaction을 받을 수 있는 최소 cycle 간격
Latency	한 transaction의 input acceptance부터 result까지 걸리는 cycle
Logical address	algorithm과 software가 보는 coefficient index
Physical tag	이 설계에서 BRAM side와 compact row를 묶은 pipeline metadata
Pipeline stage	register로 나뉜 한 cycle의 조합 연산 구간
Post-route	placement와 routing을 마친 구현 단계. 최종 timing/area 판단에 사용
Post-synthesis	논리 합성과 technology mapping 직후 단계. inference 확인에 유용
Route delay	배치된 cell 사이의 programmable interconnect 지연
Setup slack	required time에서 arrival time을 뺀 값. 음수면 setup violation
Throughput	단위 시간에 처리하는 transaction 수. latency와 별도 지표다.
TNS	음수 setup slack의 합. violation의 전체 규모를 나타낸다.
WHS	가장 작은 hold slack
WNS	가장 작은 setup slack

B. 자료와 재현 근거

B.1 AMD/Xilinx 문서

- UG901, Vivado Design Suite User Guide: Synthesis: RAM/DSP/SRL coding과 synthesis option
- UG949, UltraFast Design Methodology Guide: timing path 분석, pipelining, fanout, control set, inference
- UG906, Vivado Design Suite User Guide: Design Analysis and Closure Techniques: timing/utilization report 해석
- UG479, 7 Series DSP48E1 Slice User Guide: DSP48E1 data path와 internal register
- UG473, 7 Series FPGAs Memory Resources: RAMB36E1/RAMB18E1 구성
- UG474, 7 Series FPGAs Configurable Logic Block User Guide: LUT, FF, carry, slice 구조

B.2 사례 RTL 파일

파일	이 책에서 확인한 내용
MDL_XXX_ntt_core.v	BRAM command, physical tag, occupancy, butterfly/writeback alignment
MDL_XXX_layout_mapper.v	side/compact-row bijection과 고정 affine tap
MDL_XXX_bram_dp.v	true-dual-port synchronous BRAM inference template
MDL_XXX_zeta_rom.v	packed 2048×32 two-read-port twiddle memory
MDL_XXX_XXX_unifiedMUL.v	two-DSP, six-stage ML-DSA/R16 shared multiplier
MDL_XXX_unifiedBUT.v	two multiplier instance와 unified butterfly pipeline
MDL_XXX_ntt_addrngen.v	logical address schedule와 NTRU+864 special mode
MDL_XXX_ntt_engine.v	transport-independent core boundary
MDL_XXX_axis_load.v	64-bit beat의 2/4-coefficient write
MDL_XXX_axis_dump.v	finite FIFO credit와 AXI backpressure

B.3 측정 report

수치는 companion project에 보존된 다음 report를 source of truth로 삼았다.

- 최초 100 MHz OOC: reports/vivado_impl_baseline
- 200 MHz 목표판: reports/vivado_200mhz와 doc/200mhz_result.md
- 최종 세 범위: reports/memory_conflict_mc/axis, bram, mdl_only

- 최종 board 측정: `board_benchmark_150mhz_final_20260903.md/csv`

표를 다른 문서로 옮길 때는 report path, Vivado version, device, clock constraint, top-level을 함께 옮긴다. 숫자만 복사하면 이후의 설계 변경과 비교 범위를 추적할 수 없다.

B.4 개정 원칙

이 개정판은 기존의 “개념-쉬운 예-주의할 예-더 좋은 예-area-timing” 반복 구조를 제거했다. 공통 기초는 1장과 2장에 한 번만 설명하고, 이후 내용은 실제 RTL의 문제 경로와 변경 근거에 연결했다. 일반 규칙은 해당 사례에서 확인된 범위 안에서만 결론으로 제시한다.